

Kickoffs-MEAN-Apply_My_Course

Problem Statement

Note

Use Angular for the front end and Node.js + MongoDB for the back end. Follow the route names, field names, status codes, and response messages exactly. Hidden tests are typically strict about small mismatches.

Application Overview

Build a full-stack application that allows users to browse course cards, enroll in courses, rate courses, and drop courses. The application should fetch data from the backend and update the UI based on the course status.

FRONT-END: Angular

- This application uses Angular as the front end. Fix the test case errors in the application. The test cases are written in Jasmine.
- Use fetch API or Angular HttpClient exactly as expected by the starter project. Do not change the core route names or payload shapes.

App Component

- Write a function to get the course details from the backend using handleGetData.
- Display the course details in cards.
- Write a function to apply for a course using handleApp.
- Allow the user to add a rating only if the user has already applied for the course.
- Use the isRated property to disable the Add Rating option after rating is submitted.
- Use handleRating to store the selected rating value in component state.
- Use handleAddRating to submit the rating when the Add button is clicked.
- Write a function to drop a course using handleDrop.

BACK-END: NodeJS

- Use Node.js and MongoDB for the back end.
- learning-app-easy is the name of the database used in the application.
- There is a single collection called courses inside the database.

COLLECTIONS

There is a single model file named courses.js inside Nodejs/src/mongoose/models. The schema of the collection is as follows.

Sl. No.	Field name	Type	Validations	Required	Default
1	_id	ObjectID	Auto-generated	-	-

2	courseName	String	Should not contain any special characters except dots or spaces and length should not be less than 3	TRUE	-
3	courseDept	String	Can only have one of: WD, AI, DS, CS, CC, UI, GD	TRUE	-
4	description	String	Should have at least three words	TRUE	-
5	duration	Number	Minimum = 1, Maximum = 100	TRUE	-
6	isApplied	Boolean	-	FALSE	False
7	isRated	Boolean	-	FALSE	False
8	noOfRatings	Number	-	FALSE	0
9	rating	Number	-	FALSE	0
10	__v	Number	Auto-generated	-	0

ROUTERS

There is a single router file named users.js inside Nodejs/src/routers. The endpoints and their expected behaviors are given below.

1) /courses/enroll/:id -> POST Method

This route should enroll the user in the course whose _id matches the id from the request URL by setting isApplied to true.

If the user is enrolled successfully, return response code 200 with:

```
{ "message": "You have successfully enrolled for the course" }
```

If the user has already enrolled in the course (that is, isApplied is already true), return response code 403 with:

```
{ "error": "You have already applied for this course" }
```

If something goes wrong while executing the request, return response code 400.

2) /courses/drop/:id -> DELETE Method

This route should drop the user from the course whose _id matches the id from the request URL by setting isApplied to false.

If the drop is successful, return response code 200 with:

```
{ "message": "You have dropped the course" }
```

If the user has not enrolled in the course (that is, isApplied is already false), return response code 403 with:

```
{ "error": "You have not enrolled for this course" }
```

If something goes wrong while executing the request, return response code 400.

3) /courses/get -> GET Method

This route should fetch all records from the courses collection.

- If data is fetched successfully, return response code 200.
- If data fetching is unsuccessful, return response code 400.

4) /courses/rating/:id -> PATCH Method

This route should update rating and noOfRatings for the course whose `_id` matches the id from the request URL.

- Increment noOfRatings by 1.
- Update the previous rating based on the rating received in the request body and the previous noOfRatings.
- Round the updated rating to one decimal place.
- Set isRated to true after rating is added.

Steps to update the rating:

- Multiply the previous rating by the previous noOfRatings.
- Add the rating value received from the request body.
- Divide the total by previous noOfRatings + 1.

Sample request: `/courses/rating/5ff4264d9a608c280d745c3c`

Sample request body:

```
{ "rating": 4 }
```

If the rating is updated successfully, return response code 200 with:

```
{ "message": "You have rated this course" }
```

If the user has already rated the course (isRated is already true), return response code 403 with:

```
{ "error": "You have already rated this course" }
```

If the user has not enrolled for the course (isApplied is false) and still tries to rate it, return response code 403 with:

```
{ "error": "You have not enrolled for this course" }
```

If something goes wrong while executing the request, return response code 400.

MongoDB Commands

- Open the mongo shell by running mongo from the terminal.
- View all databases by running show dbs.
- Select the database by running use learning-app-easy.
- View collection names by running show collections.
- View collection data by running db.collection_name.find().
- Press ctrl+c to exit.